

Intermediate (test a tool without calling the LLM) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
from tools import get_weather

def test_get_weather_returns_dict():
    result = get_weather("Lahore")
    assert isinstance(result, dict)
    assert "temperature" in result

def test_get_weather_bad_city():
    result = get_weather("")
    assert result.get("error") is not None
```

This is a correct, minimal pair of tests: one for the normal case, one for bad input. No model, no API key, no network — just the function.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the tool returns an error dict on bad input (no exception)

```
from tools import get_weather

def test_get_weather_returns_expected_shape() -> None:
    result = get_weather(city="Lahore")
    assert isinstance(result, dict)
    assert "temperature" in result
    assert isinstance(result["temperature"], (int, float))

def test_get_weather_bad_city_returns_error_field() -> None:
    result = get_weather(city="")
    assert "error" in result
    assert result["error"] != ""
```

Approach 2 — the tool raises a custom error on bad input

```
import pytest
from tools import get_weather
from exceptions import InvalidToolInputError

def test_get_weather_returns_expected_shape() -> None:
    result = get_weather(city="Lahore")
    assert isinstance(result, dict)
    assert "temperature" in result
    assert isinstance(result["temperature"], (int, float))

def test_get_weather_bad_city_raises() -> None:
    with pytest.raises(InvalidToolInputError):
        get_weather(city="")
```

Difference from Basic: Basic checks the happy path plus one bad-input case with a single loose `assert ( result.get("error") is not None )`. Both Intermediate approaches split that into 2

clearly-named test functions and assert more precisely on the shape of both the success and failure results. Approach 1 keeps the tool's contract simple — it always returns a dict, and bad input just means an `"error"` key shows up instead of a crash. This matters because a tool result usually gets handed *back to the model* as the next message, and a dict is easier for the model to read than a stack trace. Approach 2 raises a real exception, which is more idiomatic Python, but means your agent's tool-calling loop needs its own `try/except` around every tool call to turn that exception back into something the model can read.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — faking a real network call with `unittest.mock.patch`

```
from unittest.mock import patch, Mock
from tools import get_weather

@patch("tools.requests.get")
def test_get_weather_success(mock_get: Mock) -> None:
    mock_get.return_value.status_code = 200
    mock_get.return_value.json.return_value = {"temp_c": 22, "condition": "Clear"}

    result = get_weather(city="Lahore")

    assert result["temperature"] == 22
    assert "error" not in result
    # prove no real network call happened
    mock_get.assert_called_once()

@patch("tools.requests.get")
def test_get_weather_api_failure(mock_get: Mock) -> None:
    mock_get.return_value.status_code = 500

    result = get_weather(city="Lahore")

    assert "error" in result
    assert "temperature" not in result # never fabricate a fake success value

@patch("tools.requests.get")
def test_get_weather_timeout(mock_get: Mock) -> None:
    import requests
    mock_get.side_effect = requests.exceptions.Timeout

    result = get_weather(city="Lahore")

    assert "error" in result
```

Expected output: all 3 pass in well under a second, with zero network access — `mock_get` replaces `requests.get` entirely for the duration of each test, and `pytest` restores the real one automatically afterward. `@patch("tools.requests.get")` patches the name as it's looked up inside `tools.py` — patching `"requests.get"` directly would miss it if `tools.py` did `from requests import get`.

Approach 2 — a separate, explicitly-marked integration test for the real API

```
import os
import pytest
from tools import get_weather

@pytest.mark.skipif(
    not os.getenv("RUN_INTEGRATION_TESTS"),
    reason="hits the real weather API – run manually with RUN_INTEGRATION_TESTS=1"
)
def test_get_weather_real_api() -> None:
    result = get_weather(city="Lahore")
    assert isinstance(result, dict)
    assert "temperature" in result
```

Expected output when run normally (`pytest`): `SKIPPED (hits the real weather API...)` — this test is invisible in everyday runs. Only `RUN_INTEGRATION_TESTS=1` `pytest` actually calls the real API, on purpose, when you want to confirm the real integration still works (e.g. after the third-party API changes its response format).

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's tests only make sense for a tool with no external dependency — if `get_weather` actually calls a real weather API, Intermediate's tests either silently hit the network (slow, costs quota, can fail for reasons unrelated to your code) or simply don't compile against the real function signature.

Approach 1 fixes that by faking the one part that isn't yours (`requests.get`), so

`test_get_weather_success`, `_api_failure`, and `_timeout` all run fast, free, and deterministically, while still exercising your real request-building and response-parsing logic.

Approach 2 is a different, complementary tool: it keeps exactly one test that hits the *real* API, but fences it off behind an env var so it never slows down or breaks a normal test run — useful for catching the day the third-party API's response shape actually changes.

Which one should you actually write? For any tool with a real network or database call, write Approach 1's mocked tests always — they belong in your normal test suite and should run on every commit. Add Approach 2's gated integration test only for tools where "the third-party API changed

shape under us" is a real, recurring risk worth catching — not for every tool, since each one adds a slow, sometimes-flaky test that a CI pipeline has to be configured to skip correctly.